

1.1 Task

Relation A)

A)

Superkeys are

EmpID

SSN

Email

EmpID, name

SSN department

Email, phone

B)

Candidate keys

EMPid

SSN

Email

C)

I would choose EMPid as the primary key, because , it is short and simple, it is specifically designed to identify an employee , it doesn't contain sensitive info, easier to use in foreign key relationships , it is stable compared with attributes such as email or phone number .

D)

Yes they can have the same phone number. The sample data shows different phone numbers , but the relation does not state that Phone must be unique. For exmpl , two employees can share a company phone or office phone.

Relation B)

The primary key should be (StudentID, CourseCode, Section, Semester, Year)

Why i chose them

Studentid - Identifies which student is registered

CourseCode- Identifies the course

Section – Identifies the specific section of the course

Semester - allows the same student to take the same course in different semesters

Year- Distinguishies the same semester across different years.

Additional candidate keys task

Based on business rules there are no candidate keys.

Task 1.2

Foreign key relations

Student.AdvisorID to Professor.ProfID

Professor.Department to Department.DeptCode

Course.DepartmentCode to Department.DeptCode

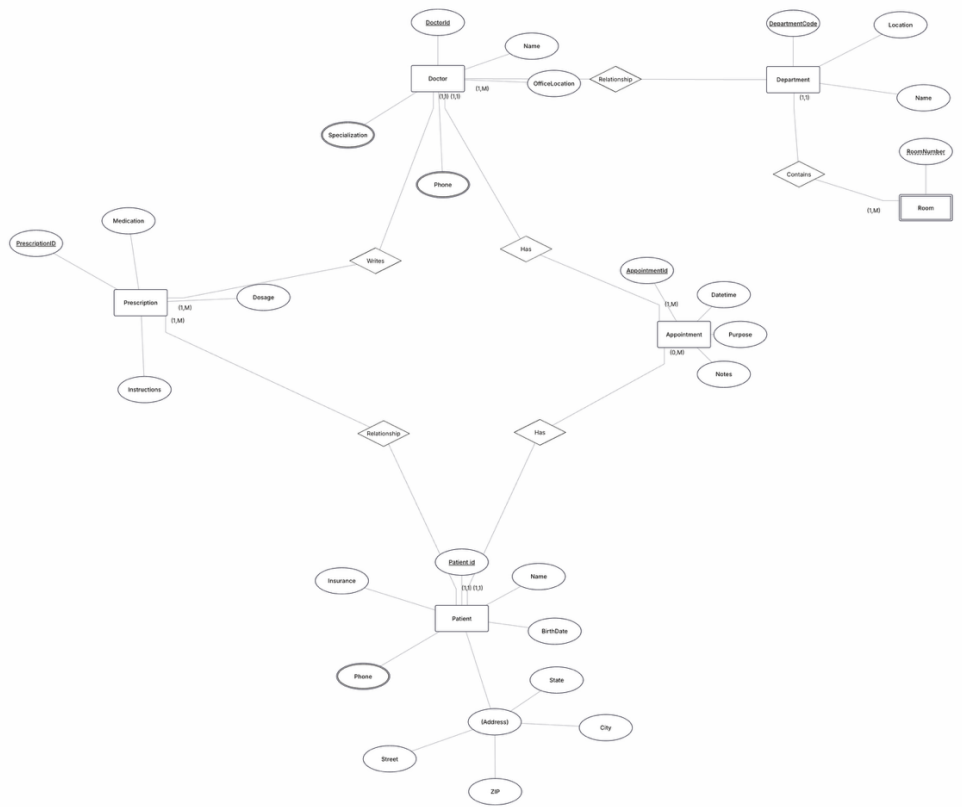
Department.ChairID to Professor.ProfID

Enrollment.StudentID to Student.StudentID

Enrollment.CourseID to Course.CourseID

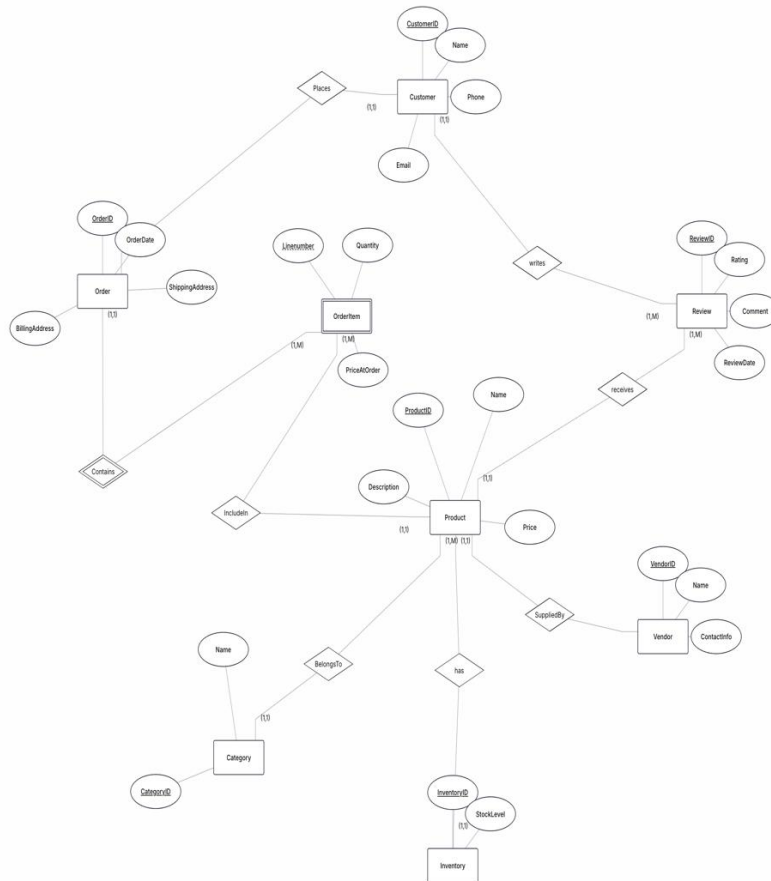
Task 2

\



weak entity is order item . OrderItem is a weak entity because it cannot be uniquely identified by itself. A line number is unique only within a specific order, so OrderItem depends on Order for its identification.

The Customer-Product relationship is many-to-many because a customer can review multiple products and a product can be reviewed by multiple customers. The relationship has attributes such as Rating, Comment, and ReviewDate.



Part 4: Normalization Workshop

Task 4.1: Denormalized Table Analysis

1. Functional Dependencies

StudentID $\rightarrow$ StudentName, StudentMajor

ProjectID $\rightarrow$ ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate

SupervisorID $\rightarrow$ SupervisorName, SupervisorDept

(StudentID, ProjectID) $\rightarrow$ Role, HoursWorked

2. Problems

Redundancy: Student and project information is repeated in multiple rows.

Update anomaly: Changing a supervisor's department requires updating multiple rows.

Insert anomaly: A new project cannot easily be added if no student is assigned to it.

Delete anomaly: Deleting the only student assigned to a project may also delete the project's information.

3. 1NF

The table is already in 1NF because all attributes contain atomic values and there are no repeating groups.

4. 2NF

Primary Key: (StudentID, ProjectID)

Partial dependencies:

StudentID $\rightarrow$ StudentName, StudentMajor

ProjectID $\rightarrow$ ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate

2NF decomposition:

Student(StudentID, StudentName, StudentMajor)

Project(ProjectID, ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate)

StudentProject(StudentID, ProjectID, Role, HoursWorked)

5. 3NF

Transitive dependency:

ProjectID $\rightarrow$ SupervisorID

SupervisorID $\rightarrow$ SupervisorName, SupervisorDept

Final 3NF decomposition:

Student(StudentID, StudentName, StudentMajor)

Project(ProjectID, ProjectTitle, ProjectType, SupervisorID, StartDate, EndDate)

Supervisor(SupervisorID, SupervisorName, SupervisorDept)

StudentProject(StudentID, ProjectID, Role, HoursWorked)

Task 4.2: Advanced Normalization

1. Primary Key

The primary key is:

(StudentID, CourseID, TimeSlot, Room)

It identifies a student's enrollment in a specific course section.

2. Functional Dependencies

StudentID $\rightarrow$ StudentMajor

CourseID $\rightarrow$ CourseName

InstructorID $\rightarrow$ InstructorName

(TimeSlot, Room) $\rightarrow$ Building

(CourseID, TimeSlot, Room) $\rightarrow$ InstructorID

3. BCNF

The table is not in BCNF because several determinants are not superkeys.

For example:

StudentID $\rightarrow$ StudentMajor

StudentID does not uniquely identify a row in the original table.

4. BCNF Decomposition

Student(StudentID, StudentMajor)

Course(CourseID, CourseName)

Instructor(InstructorID, InstructorName)

RoomSchedule(TimeSlot, Room, Building)

CourseSection(CourseID, TimeSlot, Room, InstructorID)

Enrollment(StudentID, CourseID, TimeSlot, Room)

5. Information Loss

The decomposition is lossless. The original information can be reconstructed using joins between the tables. No actual information is lost, but retrieving the complete information requires several joins.

Part 5: Design Challenge **Task 5.1: Real-World Application**

1. ER Diagram

Entities:

STUDENT: StudentID (PK), FirstName, LastName, Email

CLUB: ClubID (PK), ClubName, Description, FoundedDate

FACULTY: FacultyID (PK), FirstName, LastName, Email

EVENT: EventID (PK), EventName, Description, EventDate, StartTime, EndTime

ROOM: RoomID (PK), Building, RoomNumber, Capacity

ROOM_RESERVATION: ReservationID (PK), ReservationDate, StartTime, EndTime

BUDGET: BudgetID (PK), FiscalYear, AllocatedAmount

EXPENSE: ExpenseID (PK), ExpenseDate, Amount, Description

Relationships:

STUDENT — MEMBER_OF — CLUB

Cardinality: M:N

Relationship attribute: JoinDate

CLUB — ORGANIZES — EVENT

Cardinality: 1:M

Each club can organize zero or many events, while each event belongs to exactly one club.

STUDENT — ATTENDS — EVENT

Cardinality: M:N

Relationship attribute: AttendanceStatus

FACULTY — ADVISES — CLUB

Cardinality: 1:M

Each club has exactly one faculty advisor, while a faculty member can advise multiple clubs.

STUDENT — HOLDS_POSITION — CLUB

Cardinality: M:N

Relationship attributes: Position, StartDate, EndDate

EVENT — HAS_RESERVATION — ROOM_RESERVATION

Cardinality: 1:0..1

An event may have no reservation or one reservation, while each reservation belongs to one event.

ROOM — RESERVES — ROOM_RESERVATION

Cardinality: 1:M

A room can be reserved for many events at different times, while each reservation is for one room.

CLUB — HAS_BUDGET — BUDGET

Cardinality: 1:M

A club may have zero or many budgets, while each budget belongs to one club.

BUDGET — HAS_EXPENSE — EXPENSE

Cardinality: 1:M

A budget may have zero or many expenses, while each expense belongs to one budget.

2. Normalized Relational Schema

STUDENT(StudentID PK, FirstName, LastName, Email)

FACULTY(FacultyID PK, FirstName, LastName, Email)

CLUB(ClubID PK, ClubName, Description, FoundedDate, FacultyID FK)

MEMBERSHIP(StudentID PK, FK, ClubID PK, FK, JoinDate)

EVENT(EventID PK, ClubID FK, EventName, Description, EventDate, StartTime, EndTime)

ATTENDANCE(StudentID PK, FK, EventID PK, FK, AttendanceStatus)

OFFICER(StudentID PK, FK, ClubID PK, FK, Position, StartDate, EndDate)

ROOM(RoomID PK, Building, RoomNumber, Capacity)

ROOM_RESERVATION(ReservationID PK, EventID FK, RoomID FK, ReservationDate, StartTime, EndTime)

BUDGET(BudgetID PK, ClubID FK, FiscalYear, AllocatedAmount)

EXPENSE(ExpenseID PK, BudgetID FK, ExpenseDate, Amount, Description)

Foreign Key References:

CLUB.FacultyID → FACULTY.FacultyID

MEMBERSHIP.StudentID → STUDENT.StudentID

MEMBERSHIP.ClubID → CLUB.ClubID

EVENT.ClubID → CLUB.ClubID

ATTENDANCE.StudentID → STUDENT.StudentID

ATTENDANCE.EventID → EVENT.EventID

OFFICER.StudentID → STUDENT.StudentID

OFFICER.ClubID → CLUB.ClubID

ROOM_RESERVATION.EventID → EVENT.EventID

ROOM_RESERVATION.RoomID → ROOM.RoomID

BUDGET.ClubID → CLUB.ClubID

EXPENSE.BudgetID → BUDGET.BudgetID

3. Design Decision

There are two valid options for storing officer positions. The first option is to store Position directly as an attribute of the HOLDS_POSITION relationship. The second option is to create a separate POSITION entity with attributes such as PositionID and PositionName.

The chosen option is to store Position as an attribute of the HOLDS_POSITION relationship. This design is simpler because the system only needs to record which position a student holds in a particular club. It also allows different students to hold different positions in different clubs. StartDate and EndDate are also stored as attributes of the officer assignment.

4. Example Queries

1. Find all students who are officers in the Computer Science Club, including their positions.

2. List all events scheduled for next week with their room reservations, including the building, room number, and event time.
3. Find all expenses for each student club and show the allocated budget and the amount spent.

